

Latticra Seal Documentation Handbook

Generated from docs/latticra-seal Markdown sources.

README.md

Latticra Seal Documentation

Latticra Seal is the verification, reporting, and policy-boundary layer inside the Latticra ecosystem.

It is designed to help Latticra describe, inspect, and verify local project/system state through bounded evidence such as manifests, hashes, receipts, policy checks, status reports, and no-effect runtime-boundary metadata.

Latticra Seal is not a separate production security product. It is not currently a malware prevention system, ransomware prevention system, sandbox, kernel enforcement layer, root installer, network authority, or runtime enforcement authority.

Current role

Latticra Seal currently serves as:

- a local integrity and evidence surface
- a report-only verification lane
- a policy regression target
- a manifest/hash baseline mechanism
- a bridge between Latticra CLI, Latticra Panel, and future runtime-boundary work

Documentation map

| Document | Purpose |

|---|---|

| `STATUS.md` | Current readiness and evidence status |

| `ARCHITECTURE.md` | Internal structure and subsystem model |

| `USAGE.md` | User-facing command guide |

| `POLICY.md` | Policy-denial and fail-closed behavior |

| `REPORTS.md` | Report format and interpretation |

| `BOUNDARIES.md` | Non-claims and authority limits |

| `ROADMAP.md` | Future implementation path |

Canonical boundary

Seal documentation must remain evidence-bound.

Do not claim:

- production security readiness
- runtime enforcement
- network authority

- root authority
- kernel integration
- SELinux integration
- systemd enforcement
- Fedora approval
- malware or ransomware prevention
- broad host protection

until those claims are implemented, tested, and documented with reproducible evidence.

STATUS.md

Latticra Seal Status

Status: early evidence-bound verification layer

Scope: local report-only verification, manifest/hash baseline, policy regression, and Panel integration planning.

Current classification

Latticra Seal is currently a bounded local evidence subsystem.

It may describe local state, generate reports, compare hash baselines, and participate in policy-denial tests.

It must not be described as a production enforcement layer.

Current evidence

Known current evidence includes:

- `latticra-seal` command wrapper installed in the local user path
- local Fedora Workstation report generation
- report-only mode
- installed component marker for `latticra-seal`
- hash lock material in `latticra.seal.lock`
- policy regression workflow through GitHub Actions
- policy-denial testing through `make seal-policy-denials`

Current authority limits

```
network_authority=0
runtime_enforcement_authority=0
root_authority=0
kernel_modification_performed=0
systemd_modification_performed=0
selinux_modification_performed=0
production_security_product=0
```

Current readiness

```
local_report_generation=1
manifest_hash_baseline=1
policy_regression_lane=1
panel_bridge_planning=1
runtime_enforcement=0
network_operation=0
root_installation=0
kernel_enforcement=0
production_readiness=0
```

Correct public wording

Latticra Seal is an early verification and reporting layer inside Latticra. It focuses on local evidence, policy boundaries, manifest/hash baselines, and report-only system state inspection.

Incorrect public wording

Do not say that Latticra Seal protects production systems, prevents malware, prevents ransomware, enforces runtime isolation, modifies the kernel, installs system services, or provides certified host security.

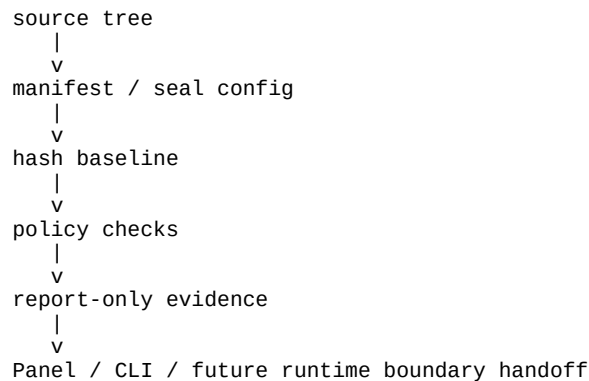
ARCHITECTURE.md

Latticra Seal Architecture

Latticra Seal is organized around bounded evidence.

It does not begin with enforcement. It begins with describing what exists, measuring what exists, reporting what changed, and refusing to claim authority it does not yet have.

Core model



Primary components

Manifest layer

The manifest layer defines what Seal is allowed to inspect.

Expected files may include:

```
latticra.seal
latticra.seal.lock
```

The manifest should describe scope, included paths, excluded paths, policy expectations, and report behavior.

Measurement layer

The measurement layer computes local evidence.

Initial measurement should remain read-only.

Expected behavior:

- read local files
- compute hashes
- compare against saved baseline
- record changed/missing/new files
- avoid mutation unless explicitly running an update/baseline command

Policy layer

The policy layer decides whether the current state is acceptable.

Initial policy behavior should be conservative:

- fail closed on malformed policy
- fail closed on denied paths
- fail closed on unsupported authority claims
- report why a decision was denied

Report layer

The report layer emits human-readable evidence.

Reports should clearly show:

- mode
- timestamp
- host OS
- prefix
- hash status
- policy status
- warnings
- failures
- non-authority declarations

Panel bridge

The Panel bridge should expose Seal through a safe interface.

Panel should be able to:

- run report-only checks
- display Seal status
- show policy results
- show generated reports
- avoid granting runtime/root/network authority

Future runtime-boundary handoff

Future runtime integration must remain gated.

Seal may eventually produce metadata for runtime-boundary decisions, but documentation must not claim runtime enforcement until the enforcement path exists and has tests.

USAGE.md

Latticra Seal Usage

This document describes the intended user-facing shape of Latticra Seal commands.

Some commands may be planned, experimental, or partially implemented. Treat the local repository and current tests as the source of truth.

Check local command availability

```
which latticra-seal
```

Generate a local report

```
latticra-seal report
```

Expected report behavior:

- report-only
- no network authority
- no runtime enforcement authority
- no root authority
- local evidence only

Run the Seal smoke lane

```
make seal
```

Run policy-denial regression checks

```
make seal-policy-denials
```

Inspect Seal lock material

```
head -40 latticra.seal.lock
```

Recommended local verification sequence

```
make seal  
make seal-policy-denials  
latticra-seal report
```

Expected documentation posture

Every command should be documented with:

- what it reads
- what it writes

- what it refuses to do
- whether it is report-only
- whether it requires root
- whether it uses the network
- what counts as PASS
- what counts as FAIL

POLICY.md

Latticra Seal Policy

Latticra Seal policy exists to define what Seal is allowed to inspect, verify, report, deny, or pass.

The policy layer should be conservative by default.

Policy goals

Seal policy should:

- make allowed scope explicit
- reject unsupported authority
- deny unsafe assumptions
- produce readable denial reasons
- support regression testing
- keep Latticra evidence-bound

Fail-closed principle

When policy input is malformed, incomplete, unsupported, or contradictory, Seal should fail closed.

Failing closed means:

```
do not silently pass
do not guess intent
do not grant authority
do not claim enforcement
emit a clear denial reason
```

Denial examples

Policy should deny:

- unknown authority modes
- network authority requests
- runtime enforcement claims
- root installation claims
- paths outside declared scope
- malformed manifests
- missing required metadata
- unsupported cryptographic claims

Policy regression

The policy regression lane should prove that unsafe or unsupported states are denied.

Expected local command:

```
make seal-policy-denials
```

Expected CI lane:

```
Latticra Seal Policy
```

Correct policy wording

Seal policy verifies whether local evidence matches declared expectations.

Incorrect policy wording

Seal policy does not currently enforce runtime behavior, block malware, isolate processes, protect the kernel, or provide production host security.

REPORTS.md

Latticra Seal Reports

Latticra Seal reports are evidence records.

A report should be readable by a human and stable enough for future tooling.

Report goals

A Seal report should answer:

- what was checked
- when it was checked
- where it was checked
- what mode was used
- what passed
- what failed
- what changed
- what authority was not used

Required report fields

Recommended fields:

```
timestamp_utc=  
mode=  
prefix=  
kernel=  
os=  
network_authority=  
runtime_enforcement_authority=  
root_authority=  
manifest_present=  
lock_present=  
hashes_checked=  
hashes_changed=  
warnings=  
failures=  
status=
```

Report modes

report-only

The default safe mode.

Report-only mode may inspect and describe local state, but should not enforce runtime behavior or mutate system state.

verification

Verification mode compares current state against declared expectations.

Verification may fail when files are missing, changed, malformed, or outside policy.

future enforcement

Future enforcement must not be claimed until implemented and tested.

PASS meaning

PASS means the checked evidence matched the declared local expectations for that command.

PASS does not mean the host is secure.

FAIL meaning

FAIL means the checked evidence did not match expectations, policy denied the state, or Seal could not safely complete the check.

FAIL should include a reason.

BOUNDARIES.md

Latticra Seal Boundaries

This document defines what Latticra Seal does not claim.

Seal is

Latticra Seal is:

- a Latticra substructure
- a local evidence layer
- a report-only verification surface
- a policy-boundary testing target
- a manifest/hash baseline mechanism
- a future bridge toward stronger runtime-boundary work

Seal is not currently

Latticra Seal is not currently:

- a production security product
- a malware prevention system
- a ransomware prevention system
- a sandbox
- a kernel module
- an SELinux policy system
- a systemd enforcement system
- a root installer
- a network authority
- a runtime enforcement authority
- a certified host protection system
- a Fedora-approved package
- an OS replacement

Required public caution

Use language like:

early
local
report-only
bounded
evidence-bound
verification-oriented

policy-regression-backed

Avoid language like:

unbreakable
production-ready
military-grade
ransomware-proof
malware-proof
kernel-enforced
runtime-enforced
certified
guaranteed secure

Rule

No claim without evidence.

No evidence without test.

No test without documentation.

ROADMAP.md

Latticra Seal Roadmap

Latticra Seal should grow in disciplined layers.

Phase 1: Documentation baseline

Goal: make Seal understandable.

Tasks:

- create documentation handbook
- define status
- define architecture
- define command usage
- define policy behavior
- define report behavior
- define non-claims

Phase 2: CLI clarity

Goal: make commands predictable.

Tasks:

- document every command
- document read/write behavior
- document exit codes
- document report schema
- document policy-denial reasons

Phase 3: Manifest and lock hardening

Goal: make local verification reproducible.

Tasks:

- stabilize manifest schema
- stabilize lock format
- define baseline update rules
- define ignored paths
- define changed-file output
- test malformed manifest behavior

Phase 4: Report schema

Goal: make reports machine-readable and human-readable.

Tasks:

- define required fields
- define optional fields
- define status values
- define warning classes
- define failure classes
- preserve report-only authority limits

Phase 5: Panel integration

Goal: make Seal visible in Latticra Panel.

Tasks:

- expose report-only scan
- expose policy-denial status
- expose report path
- expose manifest/lock status
- avoid runtime/root/network authority

Phase 6: Cryptographic strengthening

Goal: move from hash baseline toward stronger signed evidence.

Tasks:

- define signing model
- define key handling
- define trust root boundaries
- define verification receipt format
- test invalid signature behavior
- test missing key behavior

Phase 7: Runtime-boundary handoff

Goal: prepare metadata for future enforcement without claiming enforcement early.

Tasks:

- define runtime handoff metadata
- define capability gate contract

- define denial behavior
- define no-effect dry-run behavior
- test handoff reports

Phase 8: Fedora/Linux integration research

Goal: explore host integration carefully.

Tasks:

- document user-local install evidence
- document no-root assumptions
- document future RPM boundaries
- document SELinux/systemd non-claims
- document what would be required before host-level trust claims